

Pearls AQI Predictor

End-to-end serverless Air Quality Index prediction system

Project Report

Prepared for the AQI prediction project submission

1. Executive Summary

Pearls AQI Predictor forecasts AQI 24, 48, and 72 hours ahead for eight major Pakistani cities. The system combines weather and pollutant observations, feature engineering, supervised model training, local Parquet storage with optional Hopsworks integration, GitHub Actions automation, and a Streamlit dashboard.

The deployment design is deliberately resilient: the dashboard can run from the checked-in feature cache and champion model without API keys, while scheduled workflows can enrich the cache with live data when secrets are configured.

2. System Architecture

Raw weather and air-pollution data are fetched from OpenWeather for current observations. Historical pollutant observations are obtained from OpenAQ. The feature pipeline computes EPA-style AQI, calendar features, lag features, rolling statistics, and AQI change rates. Features are written to the local Parquet cache and, when explicitly enabled, to Hopsworks.

The training pipeline creates multi-output targets for 24h, 48h, and 72h horizons, compares Ridge Regression and Random Forest, optionally evaluates TensorFlow when installed, and records metrics in the model registry. Streamlit loads the champion model and latest city row to render forecasts, history, alerts, and optional SHAP explanations.

3. Feature Pipeline

The engineered schema includes hour, day, month, day of week, weekend flag, day of year, temperature, humidity, pressure, wind speed and direction, cloud cover, PM2.5, PM10, NO2, SO2, CO, O3, AQI lags at 1h/24h/72h, 24-hour rolling mean and standard deviation, and AQI change rates at 1h/24h.

Feature imputation is performed within each city so one city's history cannot fill another city's lag or rolling values. This is important for a multi-city time-series dataset.

4. Historical Backfill

The backfill workflow runs the feature-generation path over a configurable historical window and writes the resulting training rows to the feature store. The default project configuration supports up to four years, subject to OpenAQ station coverage and API limits.

5. Training Pipeline and Evaluation

The model-selection process uses a chronological train/test split to reduce leakage from future observations. Candidate models are evaluated using RMSE, MAE, and R2. The best candidate is stored as the champion and its feature schema is recorded with the artifact.

Champion model	random_forest
RMSE	37.111
MAE	28.458
R2	0.591
Forecast horizons	24h, 48h, 72h
Training rows	66104
Test rows	16527

6. Automation and Deployment

GitHub Actions provides four operational workflows: CI tests on pushes and pull requests, a manual bootstrap for historical backfill and initial training, an hourly feature pipeline, and daily model training. The workflows default to the local cache unless the `USE_HOPSWORKS` repository variable is explicitly set to true, making the default path reliable without an external feature-store account.

Streamlit Community Cloud should be configured with `app/streamlit_app.py` as the main file and Python 3.11. The checked-in `data/features/aqi_features.parquet`, `models/registry.json`, and champion artifact allow the dashboard to open before the first live pipeline run. API keys belong in Streamlit Secrets or GitHub Actions Secrets and are never embedded in source code.

7. Dashboard Features

The dashboard provides city selection, current AQI, category labels, three-day forecast bars, historical AQI charts, pollutant trends, all-city comparison, hazardous-level alerts, and SHAP feature importance. SHAP is loaded only when its tab is opened so an optional explainability failure does not prevent the forecast dashboard from starting.

8. Limitations and Future Work

Forecast quality depends on station coverage, API freshness, and the availability of complete hourly observations. The present implementation uses row-based horizons and should be upgraded to timestamp-based joins or hourly resampling when production data contain gaps. Future work should add calibrated uncertainty intervals, richer station-level spatial features, drift monitoring, and a dedicated alert delivery channel.

9. Reproduction Checklist

1. Configure `OPENWEATHER_API_KEY` and optionally `OPENAQ_API_KEY` and `HOPSWORKS_API_KEY`.
2. Run the bootstrap workflow once.
3. Confirm the CI workflow passes.
- 4.

Deploy `app/streamlit_app.py` on Streamlit Community Cloud using Python 3.11. 5. Run the hourly and daily workflows manually once to verify repository write permissions.